ORIONORDERS | ARQUITECTURA TO-BE

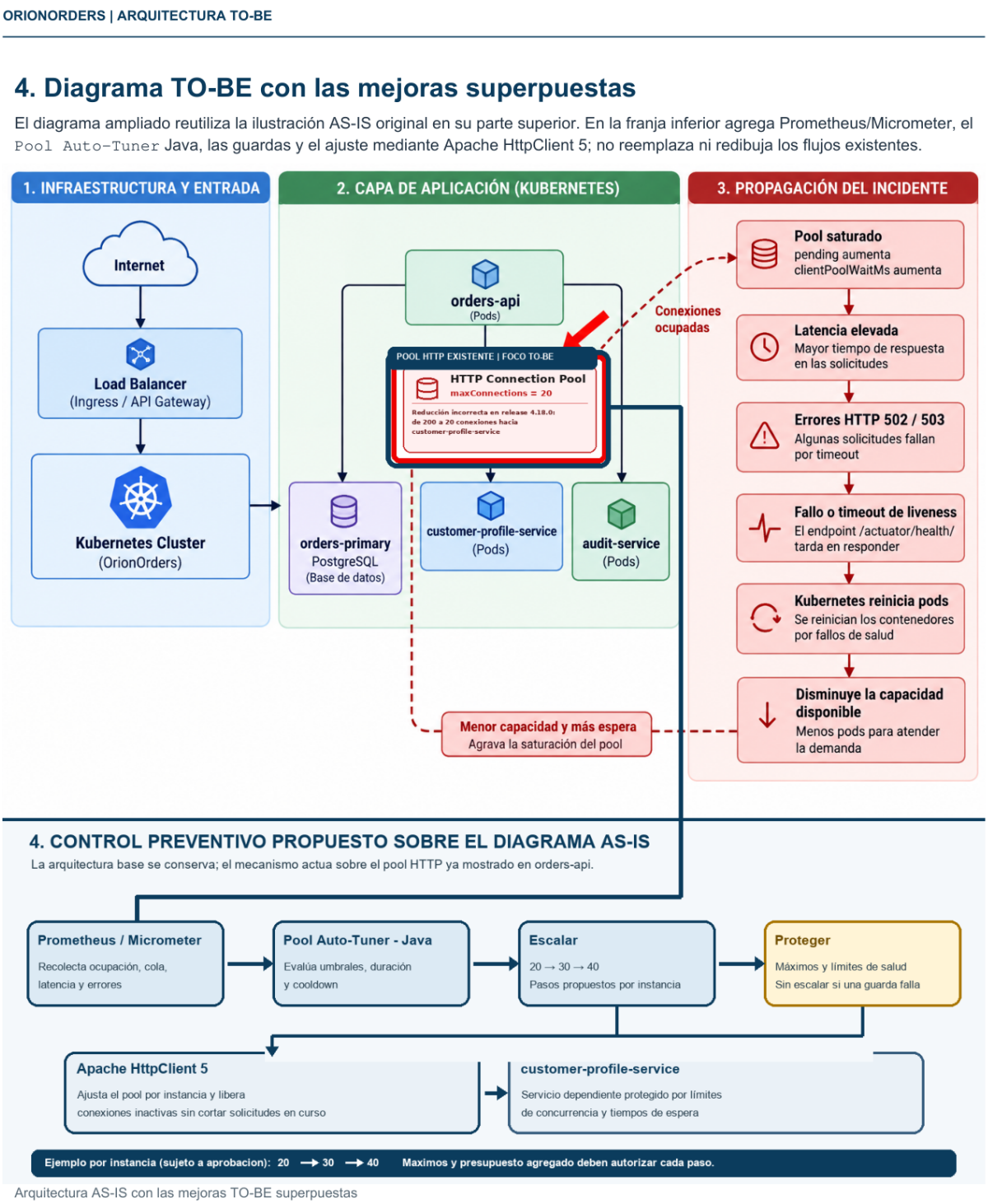
1. Objetivo y alcance

La propuesta es desarrollar un mecanismo en Java para que `orders-api` detecte la saturación de su pool HTTP, aumente la capacidad de forma controlada y gestione las conexiones inactivas sin interrumpir las solicitudes que están en curso.

El diseño propone Apache HttpClient 5 para la gestión ajustable del pool por instancia, sujeto a confirmar su versión y compatibilidad con la implementación real de `orders-api`. Conserva la llamada síncrona a `customer-profile-service` y mantiene separado el flujo asíncrono de auditoría. La adaptación no garantiza por sí sola la recuperación ni autoriza cambios productivos.

4. Diagrama TO-BE con las mejoras superpuestas

La arquitectura AS-IS se conserva y se superpone el control preventivo propuesto para la gestión del pool HTTP.



Arquitectura AS-IS con las mejoras TO-BE superpuestas

Lectura del flujo

Diccionario de herramientas

Herramientas propuestas para desarrollar, instrumentar, monitorear, desplegar y validar el mecanismo TO-BE.

1. Java + Spring Boot
Desarrollo
Para programar el Pool Auto-Tuner: revisa las métricas y decide cuándo aumentar o mantener el pool. Spring Boot se utiliza si forma parte de la tecnología de orders-api.

2. Apache HttpClient 5
Gestión de conexiones
Administra el pool HTTP, los límites por destino y el retiro de conexiones inactivas. Se debe verificar la compatibilidad con el cliente HTTP actual de orders-api.
https://hc.apache.org/httpcomponents-client-5.6.x/connection-management.html

3. Micrometer + Spring Boot Actuator
Instrumentación
Expone métricas desde Java, como conexiones ocupadas, pendientes y errores. Algunas métricas del pool requieren instrumentación personalizada.

4. Prometheus
Monitoreo
Recopila periódicamente las métricas de las instancias de orders-api para detectar presión sostenida en el pool.

5. Grafana
Visualización
Permite construir dashboards con uso del pool, solicitudes pendientes, latencia, errores HTTP 502/503 y salud del servicio dependiente.

6. Kubernetes + Helm
Despliegue y operación
Permite desplegar la solución, configurar límites y gestionar los pods. Kubernetes no aumenta por sí solo el pool HTTP: esa lógica debe implementarse en Java.

7. Maven + JUnit
Construcción y pruebas
Maven administra dependencias y compila el proyecto Java; JUnit permite probar que el controlador aumente el pool bajo condiciones correctas y respete los límites.

Nota: son herramientas propuestas. Antes de implementar, se debe confirmar la tecnología actual de orders-api, la compatibilidad de Apache HttpClient 5 y los límites aprobados para customer-profile-service.